

GridSearchCV results heatmap

Asked 2 years, 4 months ago Modified 2 years, 4 months ago Viewed 2k times



I am trying to generate a heatmap for the GridSearchCV results from sklearn. The thing I like about [sklearn-evaluation](#) is that it is really easy to generate the heatmap. However, I have hit one issue. When I give a parameter as None, for e.g.

4

```
max_depth = [3, 4, 5, 6, None]
```

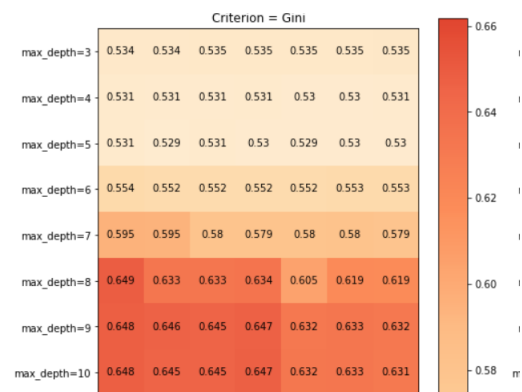


while generating, the heatmap, it shows error saying:



```
TypeError: '<' not supported between instances of 'NoneType' and 'int'
```

Is there any workaround for this? I have found other ways to generate heatmap like using matplotlib and seaborn, but nothing gives as beautiful heatmaps as sklearn-evalutaion.



[python](#) [matplotlib](#) [scikit-learn](#) [seaborn](#) [sklearn-pandas](#) [Edit tags](#)

Share Edit Follow Close Flag

asked Jun 26, 2021 at 2:19



spockshr
372 3 14

1 Answer

Sorted by: Date modified (newest first)



I fiddled around with the `grid_search.py` file `/lib/python3.8/site-packages/sklearn/evaluation/plot/grid_search.py`. At line 192/193 change the lines

5

```
row_names = sorted(set([t[0] for t in matrix_elements.keys()]),
                    key=itemgetter(1))
col_names = sorted(set([t[1] for t in matrix_elements.keys()]),
                    key=itemgetter(1))
```



To:

```
row_names = sorted(set([t[0] for t in matrix_elements.keys()]),
                    key=lambda x: (x[1] is None, x[1]))
col_names = sorted(set([t[1] for t in matrix_elements.keys()]),
                    key=lambda x: (x[1] is None, x[1]))
```

Moving all `None` to the end of a list while sorting is based on a previous [answer](#) from Andrew Clarke.

Using this tweak, my demo script is shown below:

```
import numpy as np
import sklearn.datasets as datasets
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn_evaluation import plot

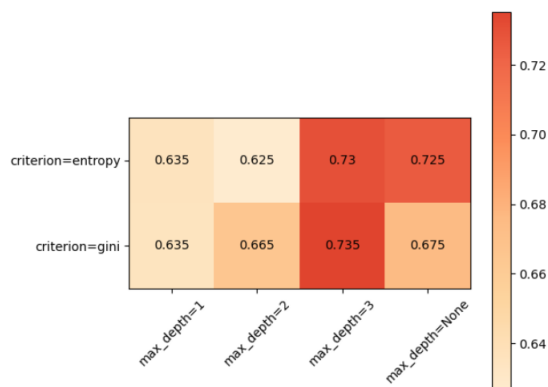
data = datasets.make_classification(n_samples=200, n_features=10, n_informative=4,
                                   class_sep=0.5)
```

```
X = data[0]
y = data[1]
```

```
hyperparameters = {
    "max_depth": [1, 2, 3, None],
    "criterion": ["gini", "entropy"],
    "max_features": ["sqrt", "log2"],
}
```

```
est = RandomForestClassifier(n_estimators=5)
clf = GridSearchCV(est, hyperparameters, cv=3)
clf.fit(X, y)
plot.grid_search(clf.cv_results_, change=(("max_depth", "criterion"), subset=
("max_features": "sqrt")))
```

```
import matplotlib.pyplot as plt
plt.show()
```



The output is as shown below:

Share Edit Follow Flag

answered Jun 26, 2021 at 5:51

 liferbeautiful
1,190 8 13

2

 This is a nice solution. For the time being, I am using seaborn heatmap which requires some manual data manipulation. It would be nice to see this solution merged in sklearn-evaluation. – spockshr Jun 26, 2021 at 12:19